Table of Content

Title: Web Application Security Assessment Report

“Damn Vulnerable Web Application (DVWA)” [LOW]

Author : Tejas G. Kamble

Github : https://github.com/TeJaS355

Purpose : Personal Security Practice and Skill Development.

Date : February, 2026

Platform : Kali Linux, Docker, DVWA

1. Environment :

OS : Kali Linux

Application : Damn Vulnerable Web Application (DVWA)

Deployment : Docker

Web Server : Apache

Database : MariaDB

Security level : low

Attacker IP : 127.0.0.1

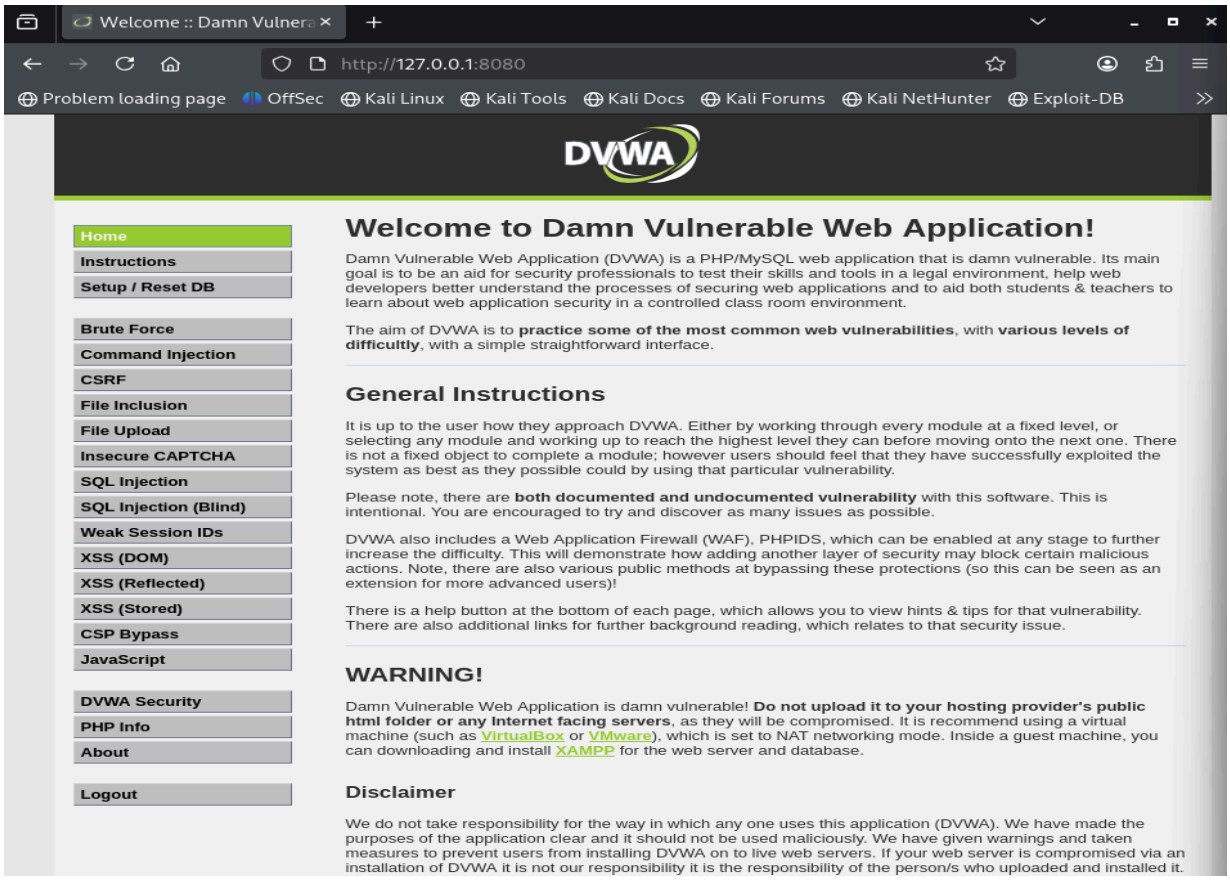

Fig.Home page of DVWA

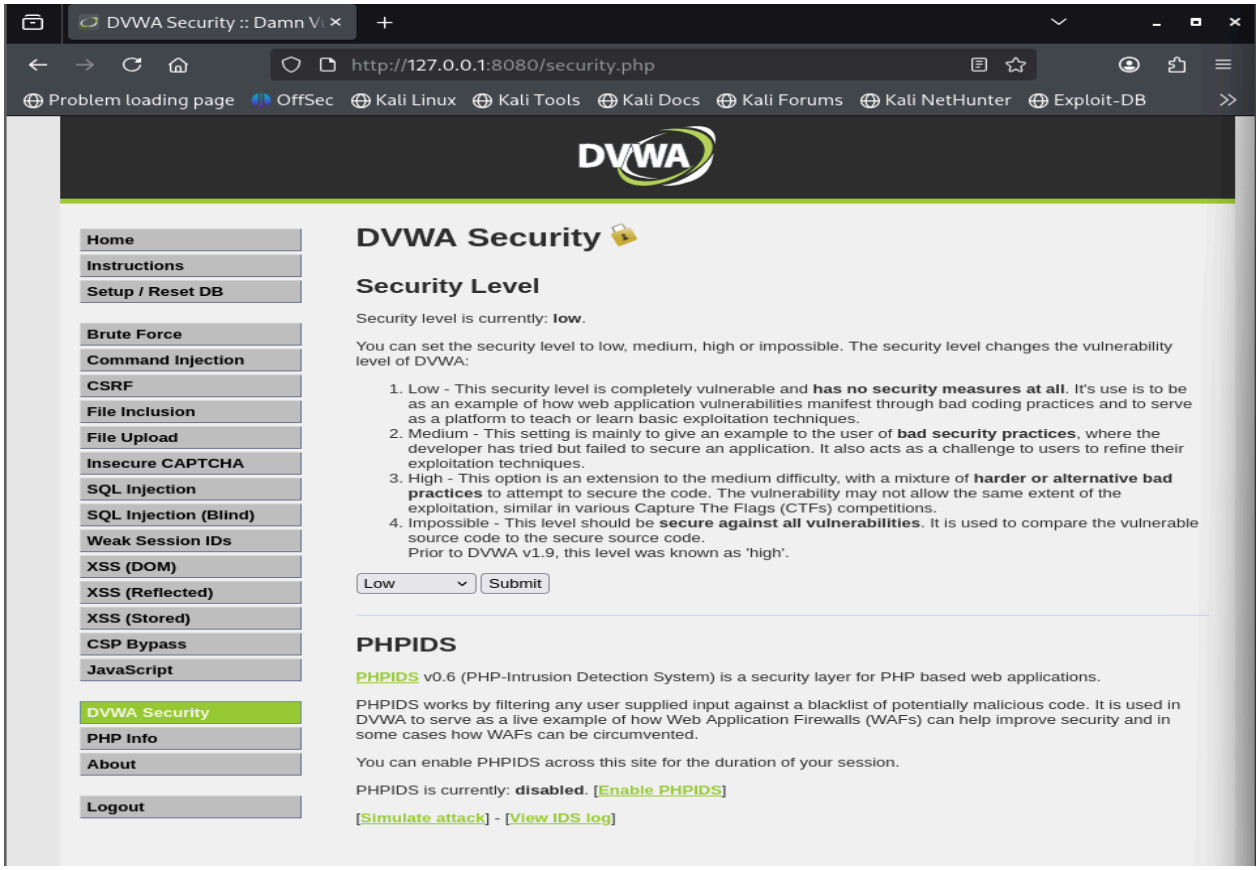

Fig. Security level of DVWA

2. Methodology :

The security assessment was conducted using a black-box testing approach against the Damn Vulnerable Web Application (DVWA). Manual testing techniques were primarily used to identify and exploit common web application vulnerabilities. Each vulnerability was tested individually using crafted payloads to observe application behavior. Successful exploitation was confirmed through application responses and visual evidence.

3. Findings :

4.1 SQL Injection :

Description

SQL Injection is a vulnerability that occurs when user input is improperly handled and directly embedded into SQL queries. This allows attackers to manipulate database queries and retrieve unauthorized data.

Affected Component :
DVWA - SQL Injection Module

Affected URL :
http://localhost:8080/vulnerabilities/sqli/

Payload used : 1' OR '1'='1

Impact :
Successful exploitation allowed the retrieval of all database records, demonstrating the potential for sensitive data disclosure and authentication bypass.

Evidence :
The application returned multiple user records after submitting the malicious payload, confirming the SQL Injection vulnerability.

Severity : High

Mitigation :

- Use Parameterized SQL queries
- Implement server-side input validation
- Restrict database user privileges

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.2 Reflected XSS :

Description

Reflected Cross-Site Scripting (XSS) is a vulnerability that occurs when user-supplied input is immediately reflected in the server's HTTP response without proper validation or output encoding. In DVWA, the Reflected XSS module processes user input and displays it back to the user, allowing an attacker to inject and execute arbitrary JavaScript code in the victim's browser.

Affected Component :
DVWA - Reflected Cross-Site Scripting Module

Affected URL :
http://localhost:8080/vulnerabilities/xss_r/

Impact :
Successful exploitation of this vulnerability may allow an attacker to :

- Execute arbitrary JavaScript in the victim's browser
- Steal session cookies
- Perform phishing or redirection attacks
-

Root Cause : The application fails to properly sanitize or encode user input before reflecting it in the HTTP response.

Severity : Medium

OWASP Category : A07:Cross-Site Scripting (XSS)

Mitigation :

- Encode output using context-appropriate encoding
- Validate and sanitize all user input
- Implement Content Security Policy (CSP)

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.3 Stored Cross-Site Scripting (XSS)

Description

Stored Cross-Site Scripting (XSS) is a vulnerability that occurs when malicious user-supplied input is stored on the server and later rendered to users without proper sanitization or output encoding. In DVWA, the Stored XSS module allows an attacker to submit JavaScript code that is saved in the backend database and executed whenever the affected page is accessed.

Affected Component :
DVWA – Stored Cross-Site Scripting Module

Target URL : http://locahost:8080/vulnerablities/xss_s/

Impact :
Successful exploitation of this vulnerability may allow an attacker to:

- Execute persistent malicious scripts in users' browsers
- Steal session cookies and authentication tokens
- Perform phishing or malicious redirection attacks

Root Cause :

The application fails to properly validate and sanitize user input before storing it and does not apply output encoding when displaying stored data.

Severity : High

OWASP Category : A07: Cross-Site Scripting (XSS)

Mitigation :

- Sanitize and validate all user input before storage
- Encode output when rendering user-controlled data
- Implement Content Security Policy (CSP)

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.4 DOM-Based Cross-Site Scripting (XSS)

Description

DOM-Based Cross-Site Scripting (XSS) is a client-side vulnerability that occurs when JavaScript code processes untrusted user input and dynamically updates the Document Object Model (DOM) without proper sanitization or encoding. Unlike reflected or stored XSS, the malicious payload is not processed by the server but is executed entirely within the victim's browser. In DVWA, the DOM XSS module uses client-side JavaScript to read user input from the URL and directly write it into the page, making it vulnerable to DOM-based XSS attacks.

Affected Component :
DVWA – DOM-Based Cross-Site Scripting Module

Target URL : http://locahost:8080/vulnerablities/xss_d/

Impact :

Successful exploitation of this vulnerability may allow an attacker to:

- Execute arbitrary JavaScript in the victim's browser
- Steal session cookies and sensitive information
- Perform phishing or malicious redirection attacks

Root Cause : The client-side JavaScript code fails to properly validate or encode user-controlled data before inserting it into the DOM.

Severity : Medium

OWASP Category : A07: Cross-Site Scripting (XSS)

Mitigation :

- Avoid using unsafe DOM manipulation methods such as "innerHTML"
- Encode and sanitize all client-side input
- Use secure JavaScript APIs such as "textContent"
- Implement Content Security Policy (CSP)

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.5 Command Injection

Description

Command Injection is a critical security vulnerability that occurs when an application executes operating system commands using unsanitized user input. This allows an attacker to inject additional system commands that are executed with the same

privileges as the vulnerable application. In DVWA, the Command Injection module accepts user-supplied input and passes it directly to system-level commands without proper validation, making it vulnerable to command injection attacks.

Affected Component :
DVWA – Command Injection Module
Target URL : http://locahost:8080/vulnerablities/exec/

Impact :
Successful exploitation of this vulnerability may allow an attacker to:

- Execute arbitrary operating system commands
- Access sensitive system files and directories
- Potentially compromise the underlying server

Root Cause :

The application fails to validate and sanitize user input before passing it to operating system commands, allowing attackers to use command separators to inject additional commands.

Severity : High

OWASP Category : A03: Injection

Mitigation :

- Avoid using system commands with user-supplied input
- Validate and sanitize all user input
- Use secure APIs that do not invoke the system shell
- Apply the principle of least privilege to application processes

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.6 Cross-Site Request Forgery (CSRF)

Description

Cross-Site Request Forgery (CSRF) is a vulnerability that forces an authenticated user to perform unintended actions on a web application without their knowledge or consent. This occurs when the application does not properly verify whether a request was intentionally made by the legitimate user. In DVWA, the CSRF module allows sensitive actions to be executed without requiring a valid CSRF token or additional user verification.

Affected Component :
DVWA – Cross-Site Request Forgery Module

Target URL : http://locahost:8080/vulnerablities/csrf/

Impact :
Successful exploitation of this vulnerability may allow an attacker to:

- Perform unauthorized actions on behalf of an authenticated user
- Change user credentials such as passwords
- Compromise user accounts without direct interaction

Root Cause :

The application does not implement CSRF protection mechanisms such as unique anti-CSRF tokens or request origin validation, allowing forged requests to be accepted by the server.

Severity : Medium

OWASP Category : A01: Broken Access Control

Mitigation :

- Implement anti-CSRF tokens for all state-changing requests

- Validate request origin and referrer headers
- Require re-authentication or CAPTCHA for sensitive actions
- Use SameSite cookies where applicable

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.7 File Upload

Description

The File Upload vulnerability occurs when an application allows users to upload files without proper validation of file type, content, or execution permissions. This can allow an attacker to upload malicious files such as web shells, which may then be executed on the server. In DVWA, the File Upload module does not adequately restrict uploaded files, making it possible to upload and execute malicious scripts on the server.

Affected Component :
DVWA – File Upload Module

Target URL : http://locahost:8080/vulnerablities/upload/

Impact :
Successful exploitation of this vulnerability may allow an attacker to:

- Upload and execute malicious scripts on the server
- Gain remote command execution
- Compromise the web server and underlying system

Root Cause : The application fails to properly validate uploaded files by extension, MIME type, and content, and stores uploaded files in a web-accessible directory.

Severity : High

OWASP Category : A03: Injection *(context-dependent) / A05: Security Misconfiguration*

Mitigation :

- Restrict allowed file types using a whitelist
- Validate file MIME types and content
- Store uploaded files outside the web root
- Rename uploaded files and disable execution permissions

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4.8 Weak Session IDs

Description

Weak Session IDs is a vulnerability that occurs when an application generates session identifiers in a predictable or insecure manner. Session IDs are used to uniquely identify a user's authenticated session. If these identifiers are weak, an attacker may be able to guess or reuse valid session IDs to hijack active user sessions. In DVWA, the Weak Session IDs module demonstrates insecure session ID generation mechanisms that can be predicted or reused by an attacker.

Affected Component :
DVWA – Weak Session IDs Module

Target URL : http://locahost:8080/vulnerablities/weak_id/

Impact :
Successful exploitation of this vulnerability may allow an attacker to:

- Hijack authenticated user sessions
- Impersonate legitimate users
- Gain unauthorized access to sensitive functionality

Root Cause : The application uses predictable or insufficiently random values when generating session identifiers, making them vulnerable to guessing or reuse.

Severity : Medium

OWASP Category : A01: Broken Access Control / A02: Cryptographic Failures

Mitigation :

- Use cryptographically secure random session identifiers
- Regenerate session IDs after authentication
- Implement proper session expiration and timeout mechanisms
- Use secure cookie attributes (HttpOnly, Secure, SameSite)

Note : The application returned all user records after submitting the malicious payload, confirming the vulnerability. Proof of exploitation is provided in Section 5.

4. Proof of Exploitation :

5.1 SQL Injection

Target URL :
http://localhost:8080/vulnerabilities/sqli/

Payload used : 1' OR '1'='1

Steps to Reproduce :
Step 1 : Navigate to the SQL Injection module in DVWA.

Step 2 : Enter the payload into the User ID input field.

Step 3 : Click the Submit button.

Result :

The application returned multiple user records from the database, indicating that the SQL query logic was successfully manipulated. This confirms that the application is vulnerable to SQL Injection.

Evidence :

Fig 5.1.1 : is shows source code the dashboard functionality.

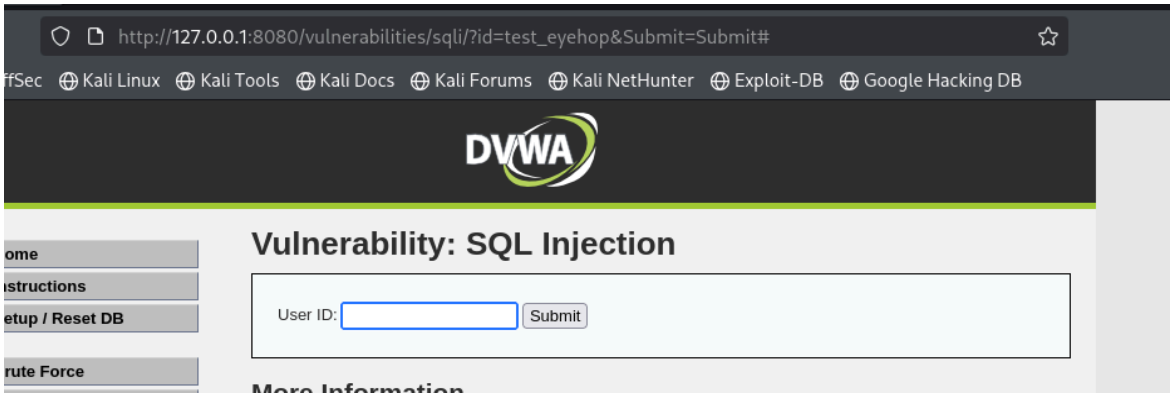

Fig 5.1.2 : shows in URL that what it get after submitting any username like in ?id=<userid> it get submitted without any authentication.

Fig 5.1.3 : shows after submitting the payload in "?id=<payload>" it get compromise and shows all data of database like id, first name and Surname.

5.2 Reflected Cross-Site Scripting (XSS)

Target URL : http://localhost:8080/vulnerabilities/xss_r/

Payload used : <script>alert("test");</script>

Steps to Reproduce :

Step 1 : Navigate to the Reflected XSS module in DVWA.

Step 2 : Enter the payload into the input field.

Step 3 : Click the Submit button.

Result :

The injected JavaScript payload was reflected back in the server response and executed in the browser, displaying a JavaScript alert dialog. This confirms the presence of a Reflected Cross-Site Scripting vulnerability.

Evidence :

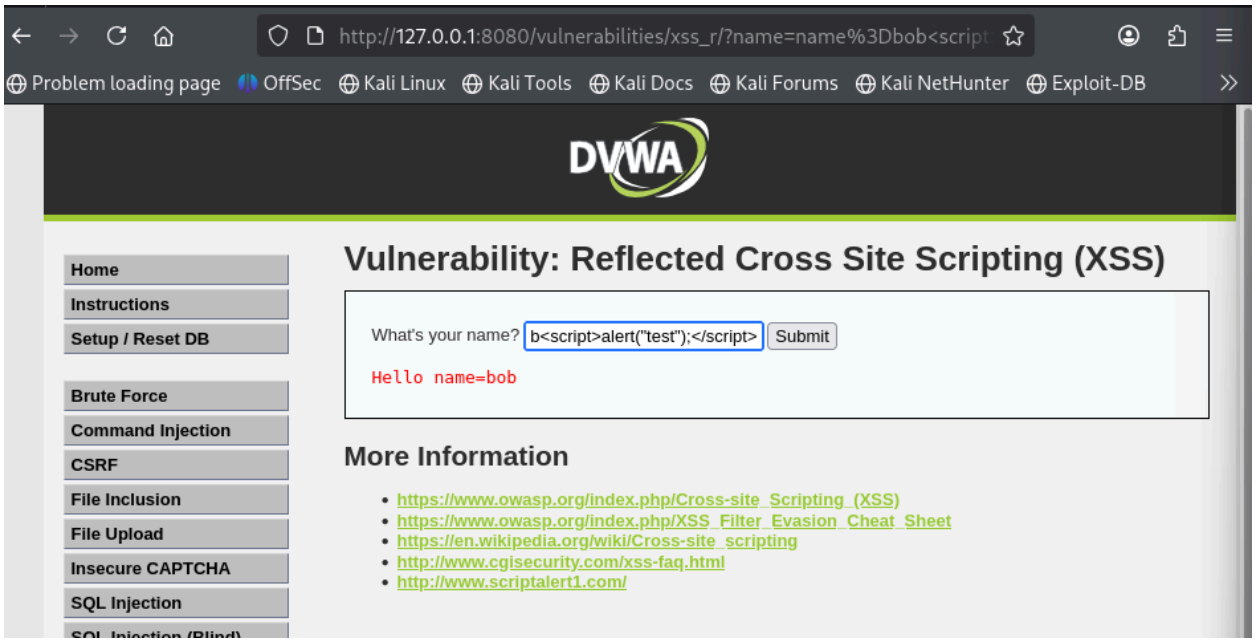

Fig 5.2.1 : Reflected XSS payload submitted through the input field

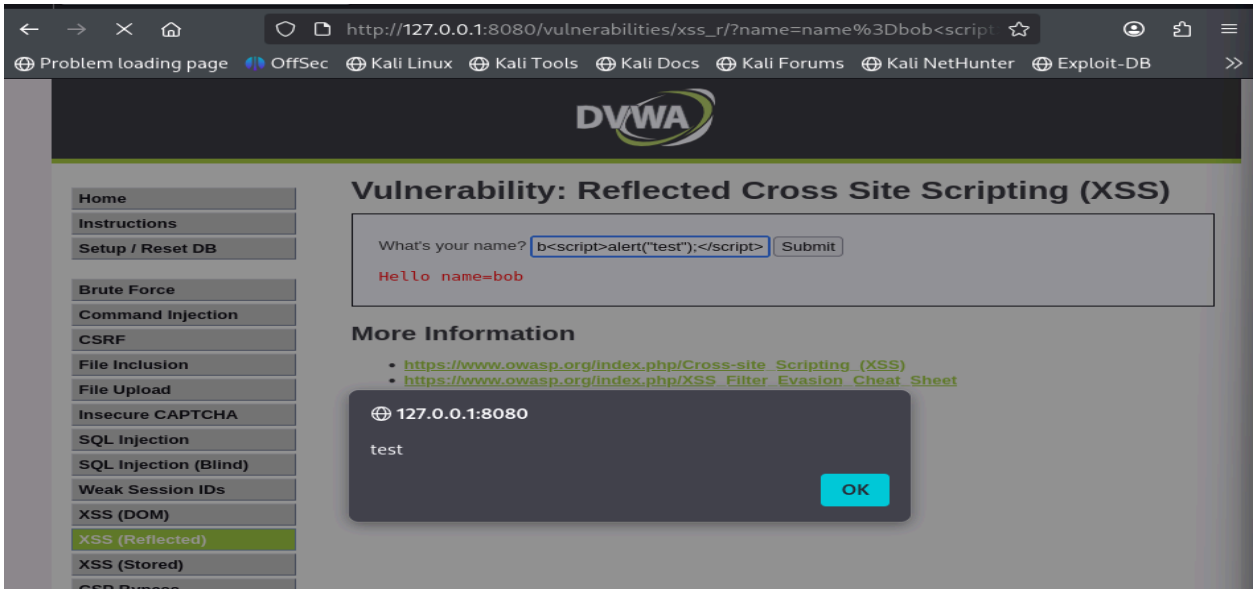

Fig 5.2.2 : JavaScript alert executed in the browser.

5.3 Stored Cross-Site Scripting

Target URL : http://localhost:8080/vulnerabilities/xss_s/

Payload Used : <script>alert(“TEST”);</script>

Steps to Reproduce :

Step 1 : Navigate to the Stored XSS module in DVWA.

Step 2 : Enter the payload into the message input field.

Step 3 : Submit the form and reload the page.

Result :

The injected script was stored on the server and executed each time the affected page was accessed. This confirms the presence of a Stored Cross-Site Scripting vulnerability.

Evidence :

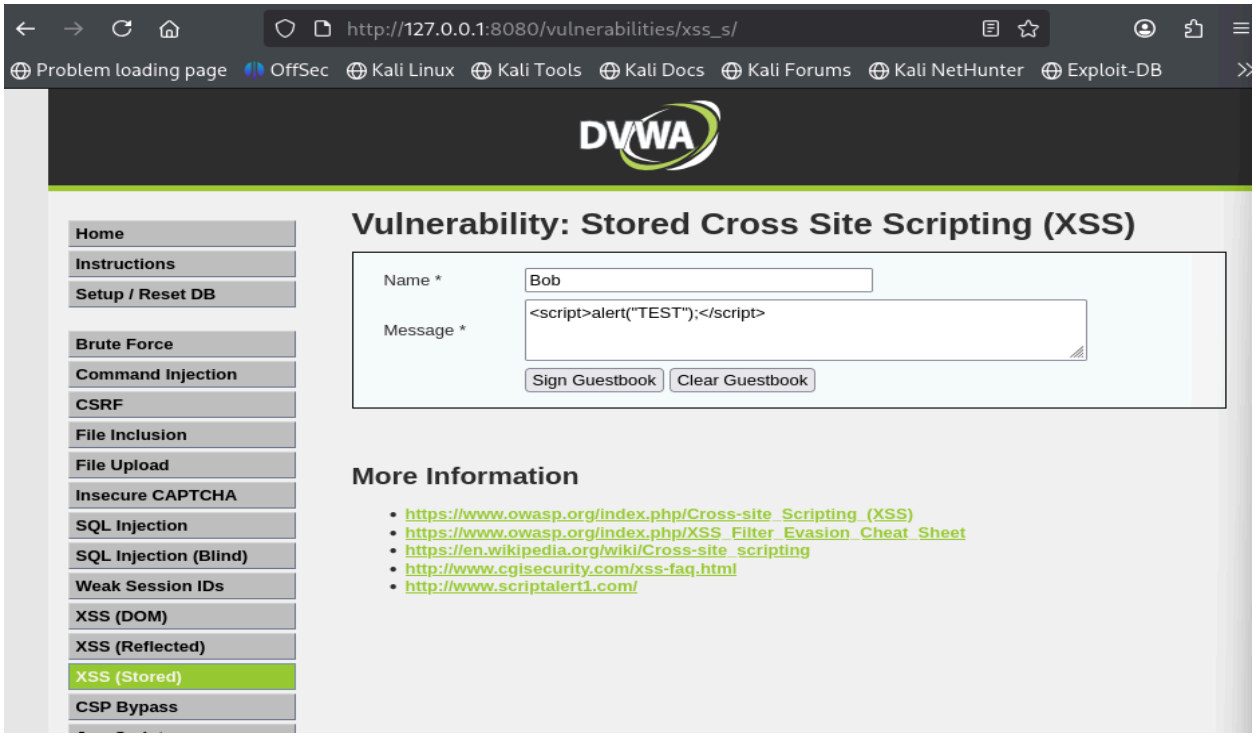

Fig. 5.3.1 : Stored XSS payload submitted

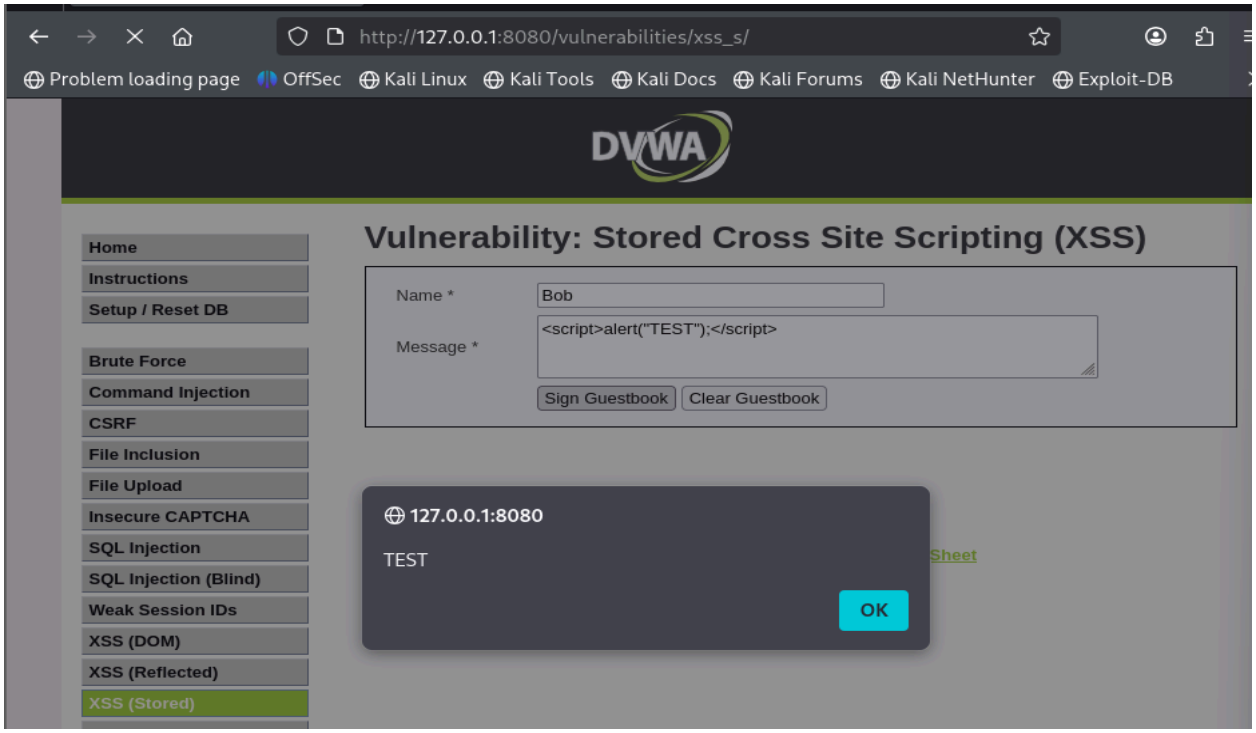

Fig. 5.3.2 : Persistent JavaScript execution upon page reload.

5.4 DOM-Based Cross-Site Scripting (XSS)

Target URL : http://localhost:8080/vulnerabilities/xss_d/

Payload used : <script>alert(“TEST”);</script>

Steps to Reproduce :

Step 1 : Navigate to the DOM-Based XSS module in DVWA.

Step 2 : Append it to the URL parameter.

Step 3 : Load the modified URL.

Result :

The malicious JavaScript payload was executed in the browser as a result of unsafe client-side DOM manipulation. The script executed without being processed by the server, confirming the presence of a DOM-Based Cross-Site Scripting vulnerability.

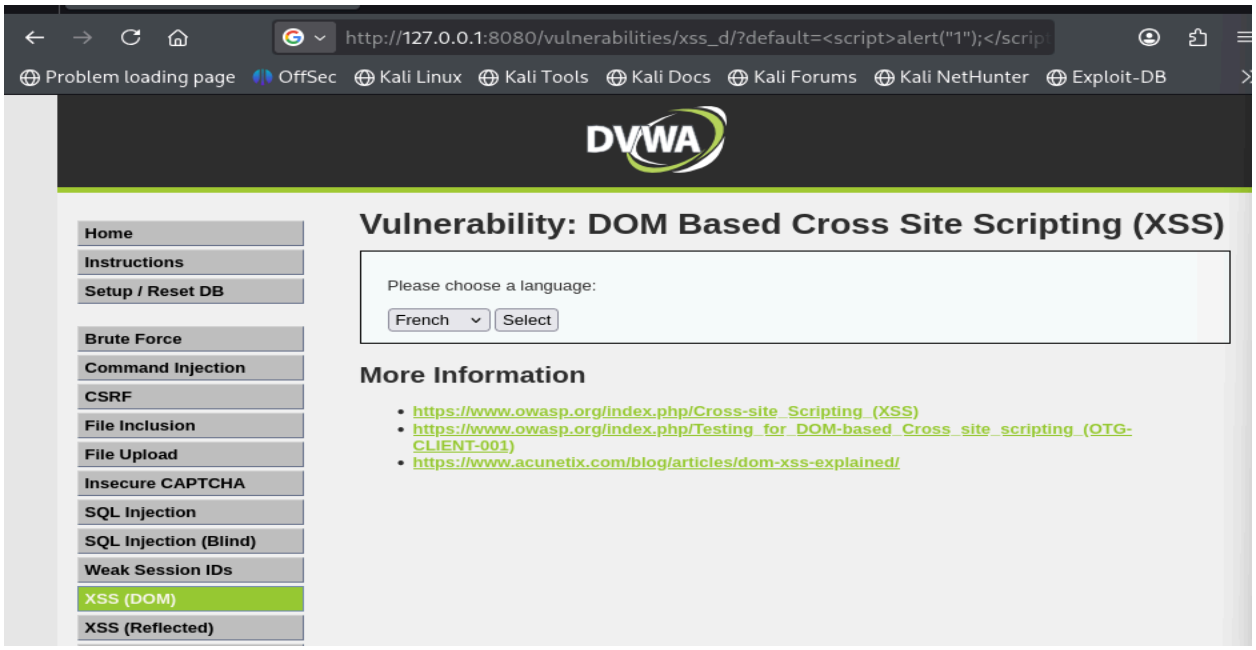

Fig. 5.4.1 : DOM XSS payload injected via URL/input field

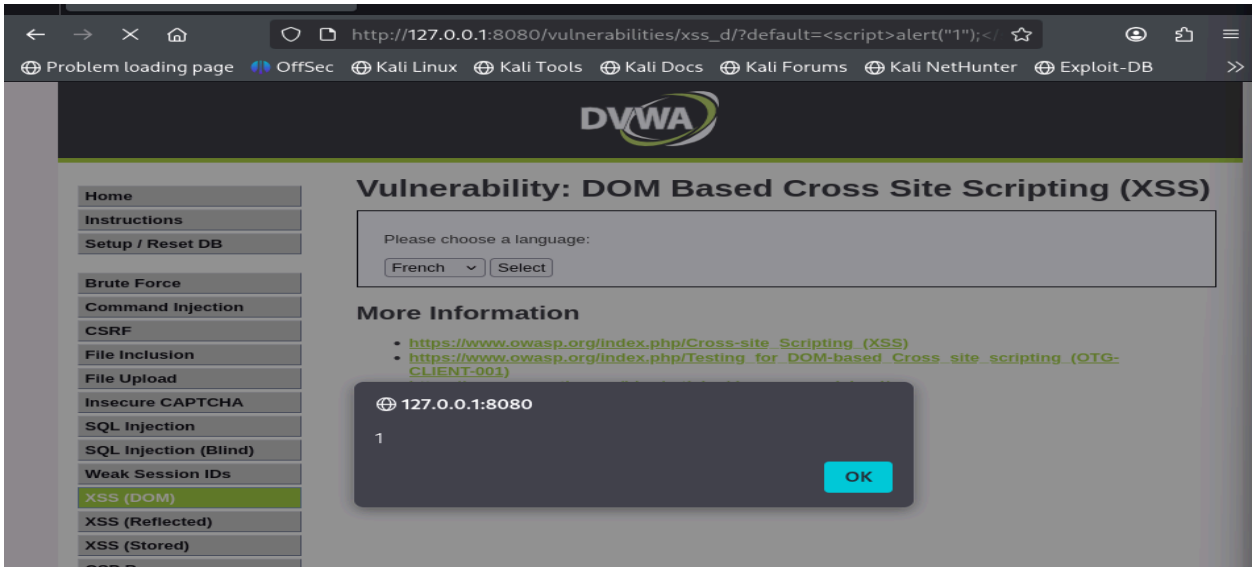

Fig. 5.4.2 : JavaScript alert executed in the browser

5.5 Command Injection

Target URL : http://localhost:8080/vulnerabilities/exec/

Payload used : 8.8.8.8; ls -la

Steps to Reproduce :

Step 1 : Navigate to the Command Injection module in DVWA.

Step 2 : Enter the payload into the input field (e.g., IP address field).

Step 3 : Click the **Submit** button.

Result :

The injected command was executed by the underlying operating system, and the output of the ls -la command was displayed in the application response. This confirms that arbitrary system commands can be executed through user-supplied input.

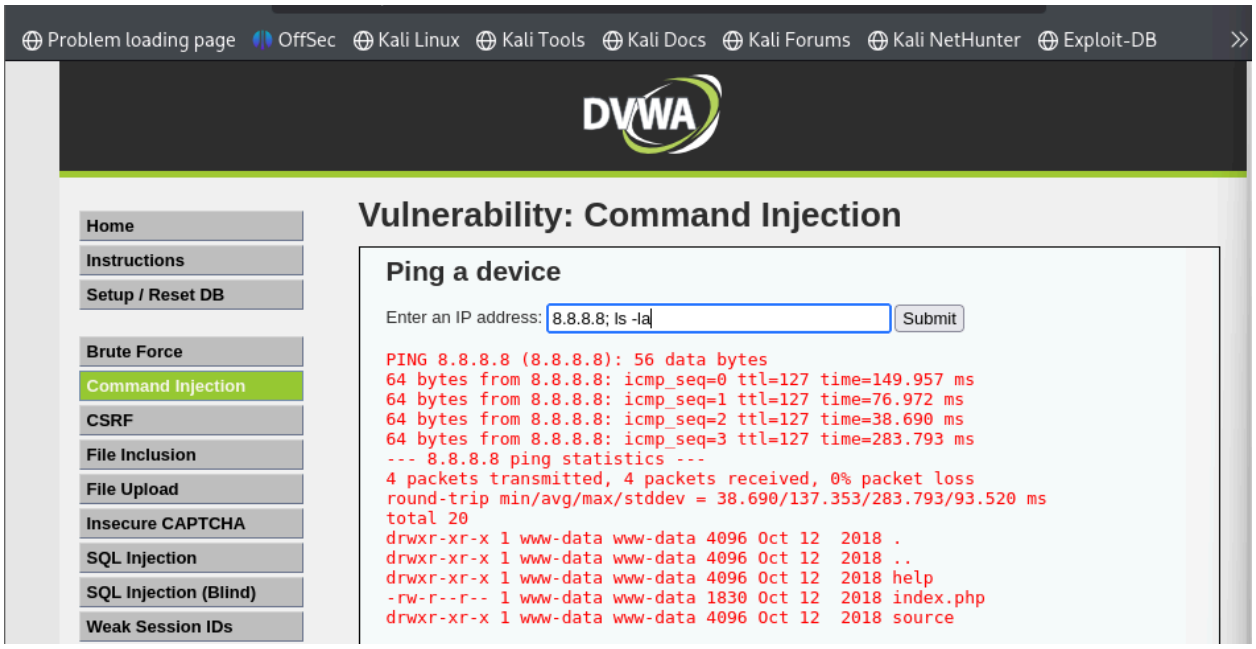

Fig. 5.5.1 : Output of injected system command displayed

5.6 Cross-Site Request Forgery

Target URL : http://localhost:8080/vulnerabilities/csrf/

Attack Scenario :

An authenticated DVWA user is tricked into visiting a malicious web page that automatically submits a forged request to the DVWA application. Since no CSRF protection is implemented, the request is processed without user consent.

Malicious Request Used :

http://localhost:8080/vulnerabilities/csrf/?password_new=test&password_conf=test&Change=Change

Steps to Reproduce :

Step 1 : Log in to DVWA as a valid user.

Step 2 : Navigate to the CSRF module.

Step 3 : While logged in, open a new browser tab or crafted HTML page containing the malicious request URL.

Step 4 : Access the malicious link or page.

Result :

The user's password was changed successfully without any confirmation or verification from the user. This confirms that the application is vulnerable to Cross-Site Request Forgery.

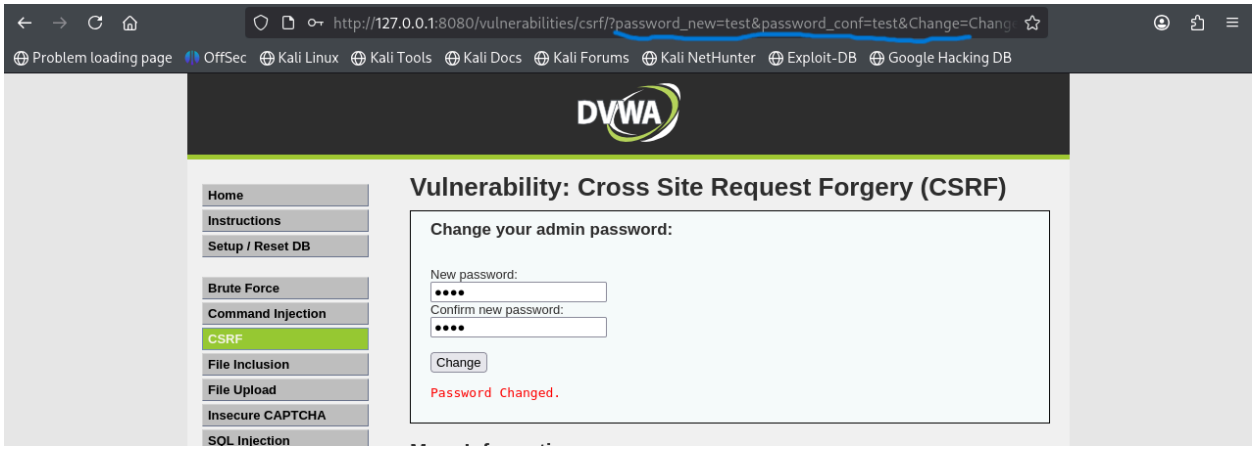

Fig. 5.6.1 : Password successfully changed without user interaction.

5.7 File Upload

Target URL : http://localhost:8080/vulnerabilities/upload/

Attack Scenario :

An authenticated DVWA user is tricked into visiting a malicious web page that automatically submits a forged request to the DVWA application. Since no CSRF protection is implemented, the request is processed without user consent.

Malicious File Used :

A PHP web shell file named **"test.php"** with the following content:

```
<?php system($_GET['cmd']); ?>
```

Steps to Reproduce :

Step 1 : Navigate to the File Upload module in DVWA.

Step 2 : Upload the malicious **"test.php"** file using the file upload form.

Step 3 : After successful upload, access the uploaded file via the browser.

Step 4 : Append a system command as a URL parameter.

Exploitation URL Example :

http://localhost:8080/hackable/uploads/shell.php?cmd=whoami

Result :

The uploaded PHP file was successfully executed on the server, and the output of the injected system command was displayed in the browser. This confirms that arbitrary file upload and remote command execution are possible.

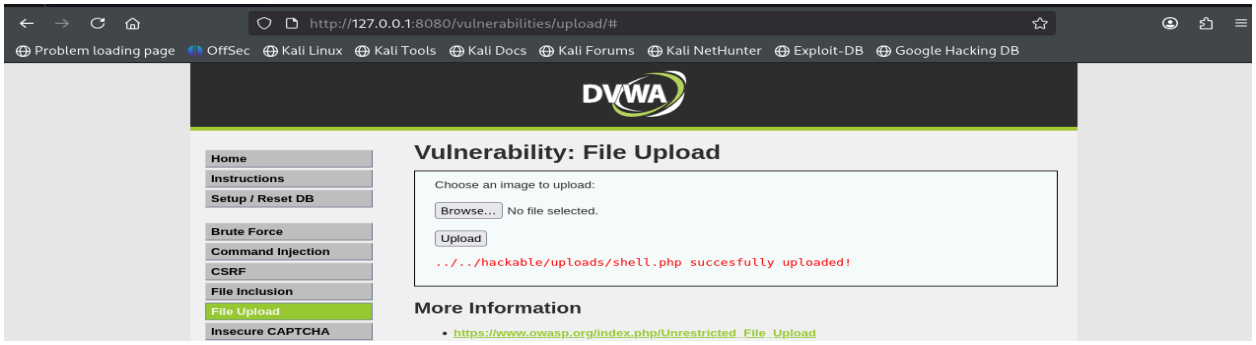

Fig. 5.7.1 : Malicious file uploaded successfully.

5.8 Weak Session IDs

Target URL : http://localhost:8080/vulnerabilities/weak_id/

Attack Scenario :

An attacker observes that session identifiers generated by the application follow a predictable pattern. By incrementing or reusing a session ID value, the attacker can gain access to another user's session.

Steps to Reproduce :

Step 1 : Log in to DVWA as a valid user.

Step 2 : Navigate to the Weak Session IDs module.

Step 3 : Observe the session ID value generated by the application.

Step 4 : Log out and refresh the page or generate a new session ID.

Step 5 : Notice that the session ID follows a predictable pattern (e.g., sequential values).

Step 6 : Manually modify the session ID to a previously observed value.

Result :

The application accepted the manipulated session ID and granted access without proper validation, confirming that session identifiers are weak and predictable.

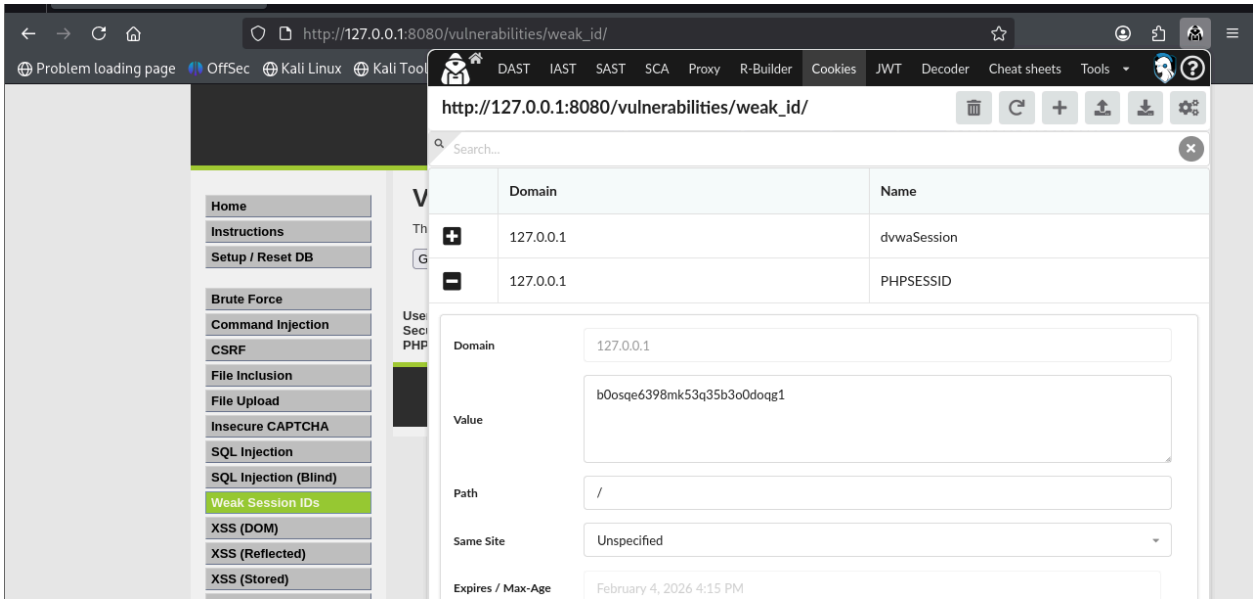

Fig. 5.8.1 : Original session ID value observed

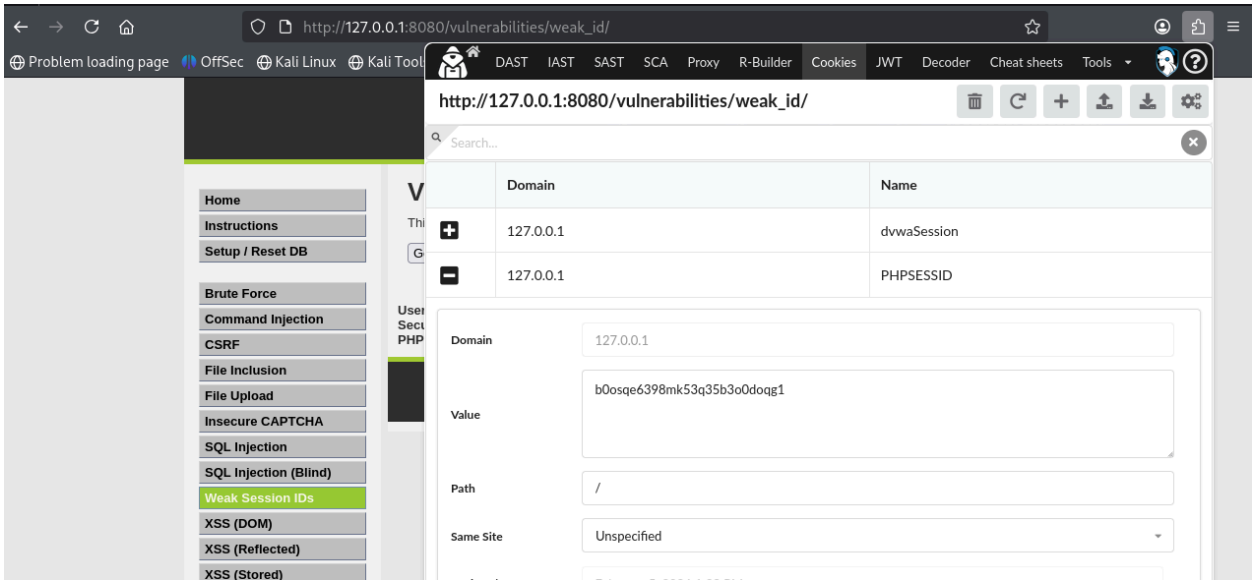

Fig. 5.8.2 : Modified session ID reused successfully.

5. Risk Summary

Vulnerability	Severity	Impact
SQL Injection	High	Database compromise
Stored XSS	High	Persistent client-side attacks
Reflected XSS	Medium	Session hijacking
DOM XSS	Medium	Client-side exploitation
Command Injection	High	Remote command execution
File Upload	High	Full server compromise
CSRF	Medium	Unauthorized actions
Weak Session IDs	Medium	Session hijacking

6. Recommendations

Based on the identified vulnerabilities, the following security recommendations are advised:

- Implement server-side input validation and output encoding
- Use parameterized queries to prevent SQL Injection
- Enforce strong session management practices

- Implement CSRF protection mechanisms
- Restrict and validate file uploads
- Apply least-privilege principles to application services
- Perform regular security testing and code reviews

7. Conclusion

The security assessment of the Damn Vulnerable Web Application revealed multiple critical and high-risk vulnerabilities, including SQL Injection, Command Injection, and insecure file upload handling. These vulnerabilities demonstrate common security weaknesses that can lead to full system compromise if present in real-world applications. The findings highlight the importance of secure coding practices, proper input validation, and robust session management to protect web applications from exploitation.

8. References

- OWASP Top 10 – https://owasp.org
- DVWA Documentation
- OWASP WebGoat Project
- PHP Security Best Practices